# *Exercise 2: Audio Visualisation with Meyda and a p5.speech Voice Controller*

*(words: 499)*

*David Cardozo*

*5 July 2026*

Both applications are p5.js sketches built on one idea: measured properties of the audio, not hand-animated timelines, drive every visual variable.[1] This report describes what is mapped and why, from the standpoint of visual impact, and shows the voice controller working. All screenshots are captured live from the running applications.

[1] Meyda: audio feature extraction for the Web Audio API, https://meyda.js.org. p5.speech: Web Speech API wrappers for p5.js, https://idmnyu.github.io/p5.js-speech/.

## *Task 2.1: Sonifying the CAPTCHA*

The audio CAPTCHA presents three word–digit pairs using p5.speech. For each word, the system randomly selects a voice, pitch, and speaking rate, so repeated attempts do not sound identical. A p5.sound masking layer runs underneath the speech, combining pink noise band-passed around the speech range near 1.4 kHz with a low brown-noise rumble. The mask is also amplitude-modulated, which makes it harder for a bot recording the browser tab to subtract the background cleanly.




Figure 1: Task 2.1 mid-challenge. The LCD scope pulses with the spoken word; the trace is the band-passed masking bed, peak-normalised per frame and scaled by the word-synced envelope.

Because the browser speech-synthesis API does not expose the generated audio buffer, the spoken voice cannot be analysed directly in the sketch. Instead, the oscilloscope visualises the masking layer, which is the actual signal available in the p5.sound graph. Each p5.FFT wave-

form frame is normalised to its own peak and then scaled by an envelope that rises while a word is being spoken. As a result, the LCD display appears to pulse in sync with the speech, as shown in Figure 1.

### *Task 2.2: Feature selection and mapping*

Feature selection was measured, not guessed: an offline pass with librosa over 512-sample frames (Meyda's exact buffer size) computed how much each candidate feature actually moves in each sound.[2] The rule: map the features with the largest relative swing, because a feature that barely changes produces a static, invisible mapping. Each sound then receives a visual form that suits its measured character (table below, Figures 2–4).

[2] Coefficient of variation, CV = std/mean, per feature and sound; the full statistics are in the appendix.

| Sound (measured character) | Meyda features | Mapping, from a visual-impact standpoint |
|---|---|---|
| **1**: dark (centroid ≈ 810 Hz), tonal (flatness ≈ 0), sparse events (flux CV 0.62) | rms, spectralFlux, spectralCentroid | "Ripple pond": rings breathe with loudness; each onset launches a travelling ripple; hue drifts inside a dark blue–violet band, preserving the sound's identity |
| **2**: volatile; rms CV 1.15, centroid CV 1.26 across 10.9 kHz, flatness spans 0–1 | spectralCentroid, rms, spectralFlatness | "Particle burst field": colour runs dark cyan to hot magenta with brightness; emission rate and size explode with loudness; flatness turns laminar streams into turbulence |
| **3**: bright (centroid ≈ 4 kHz), high-pitched (zcr 0.16), rhythmic (flux CV 0.81) | rms, spectralFlux, zcr | "Spark shower": onsets fire firework bursts; rms sets spark size and shimmer density; zcr pins the amber-to-green hue signature |

All features are normalised to 0–1 and exponentially smoothed so the mappings stay stable and comparable across sounds. Meyda's streaming analyzer cannot compute spectralFlux (it lacks the previous frame), so flux is derived from consecutive amplitude spectra as rectified rising energy with an adaptive peak; a value 1.8× above its running baseline is treated as an onset. Together the mappings exercise the brief's visual variables: number of shapes, size, fill colour, opacity, background colour and rotation.

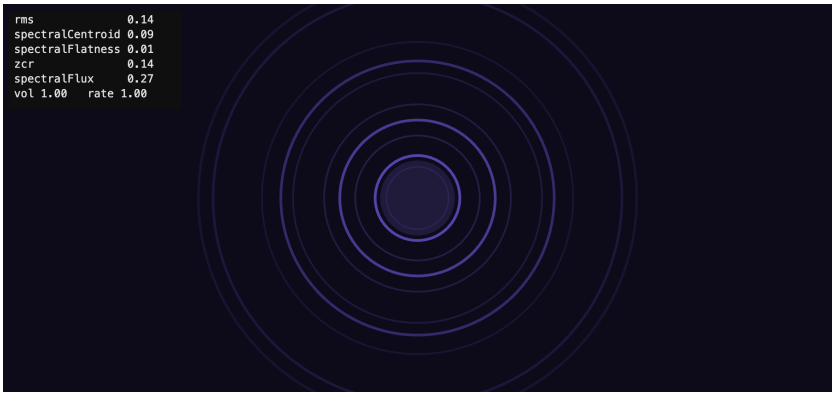



Figure 2: Sound 1, ripple pond. Rings breathe with rms; a flux onset has launched travelling ripples; the live feature readout sits top-left.



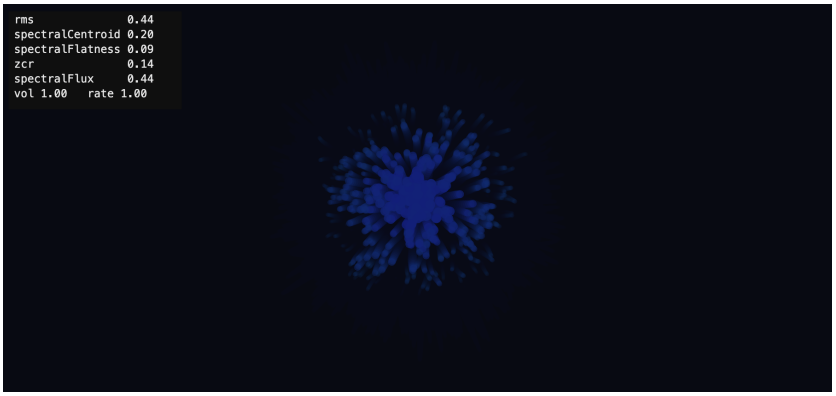




Figure 3: Sound 2 at both extremes: quiet dark-cyan wisps (top, rms 0.25) versus a loud burst of 375 particles (bottom, rms 0.44). One mapping, radically different frames: exactly why the high-CV features were chosen.

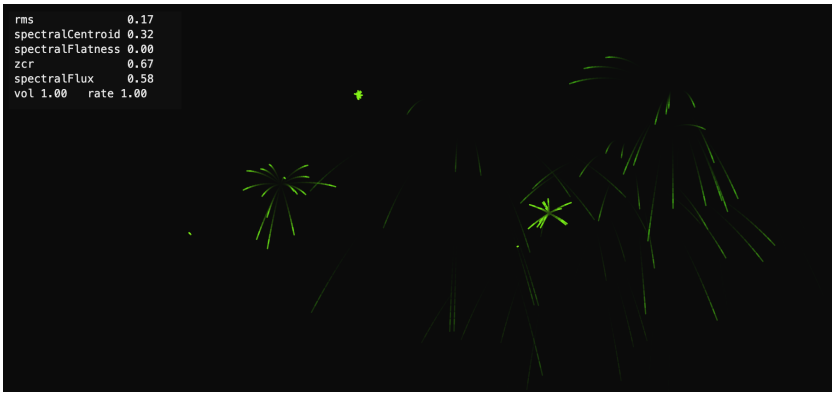



Figure 4: Sound 3, spark shower. Flux onsets fire firework bursts, rms sets spark size, and zcr (0.67 here) pins the yellow-green hue of this bright, high-pitched sound.

### *Task 2.3: Voice controller*

The voice controller runs p5.SpeechRec in continuous mode; recognition engines stop on silence, so an `onEnd` handler restarts listening while the toggle is on. Each utterance is scanned word by word and the latest keyword per category wins, with homophones accepted where recognisers commonly mishear ("sound *to/too/two*"). The required commands (five background colours and four shapes) act as an override layer on every visual, and the extended set covers transport, volume, playback rate, figure scale and trail effects:

| Category | Spoken words | Effect |
|---|---|---|
| Background | black, white, red, blue, green | overrides the visual's native backdrop |
| Shape | square, triangle, circle, pentagon | re-renders every generated figure |
| Transport | play, stop, next, "sound one…four" | track switching, homophone-tolerant |
| Volume / speed | louder, quieter, faster, slower | $\pm 0.2$ volume; $\times 1.25$ playback rate |
| Effects | bigger, smaller, trails, clean, invert, reset | figure scale, trail overpaint, brightness flip, clear all |

Applied to the *Kalte Ohren* remix, visualised as a figure field whose size pulses with rms (CV 0.54), spawns bursts on flux onsets (CV 0.63) and colours figures by centroid (CV 0.50), the recognised phrase "green triangle" repaints the background and re-renders every figure within one frame (Figure 5). A chip beside the transport echoes the last understood command, and a denied microphone cleanly shuts the listening loop down instead of retrying forever.

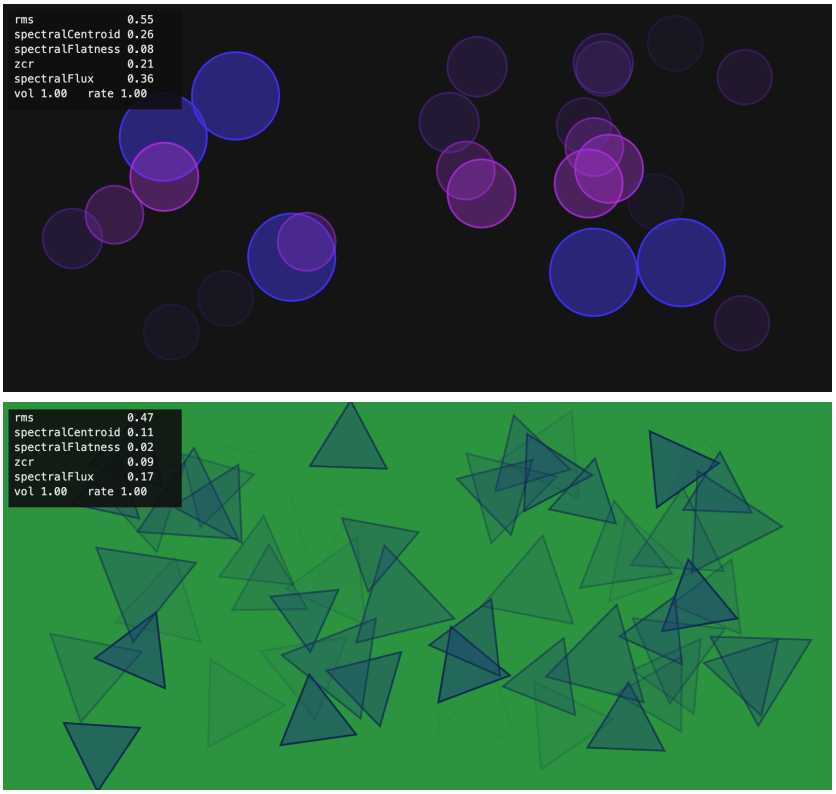



Figure 5: Task 2.3 on the Kalte Ohren remix. Native look (top); one utterance later ("green triangle") the background colour and figure shape are overridden (bottom).

## *Appendix*

### *A. Feature statistics (librosa, frame = hop = 512)*

Coefficient of variation (std/mean) per feature; the mean rows give each sound's absolute character. Flatness CVs are inflated by near-zero means and were used only as a tonal-vs-noisy indicator.

| | Sound 1 | Sound 2 | Sound 3 | Remix |
|---|---|---|---|---|
| duration (s) | 5.3 | 14.6 | 14.3 | 280.0 |
| rms CV | 0.46 | 1.15 | 0.75 | 0.54 |
| centroid CV | 0.28 | 1.26 | 0.12 | 0.50 |
| flatness CV | 4.52 | 4.14 | 2.73 | 21.95 |
| zcr CV | 0.26 | 1.13 | 0.19 | 0.88 |
| flux CV | 0.62 | 1.35 | 0.81 | 0.63 |
| mean rms | 0.052 | 0.061 | 0.057 | 0.202 |
| mean centroid (Hz) | 810 | 1786 | 3962 | 3164 |
| mean zcr | 0.028 | 0.028 | 0.159 | 0.059 |

### *B. Code excerpts*

The statistics come from `analyze.py` (librosa, frame and hop of 512 samples to mirror Meyda's `bufferSize`), run with `uv run analyze.py`. The excerpts below are from `sketch.js`.

Deriving spectralFlux from consecutive amplitude spectra, inside the Meyda analyzer callback:

```
let flux = 0;
for (let i = 0; i < spec.length; i++) {
  const d = spec[i] - prevSpectrum[i];
  if (d > 0) flux += d; // rising energy only
}
```

Adaptive flux normalisation and onset detection; a spike spawns a ripple, spark burst or figure burst depending on the sound:

```
fluxMax = Math.max(fluxMax * 0.999, rawFlux);
const fluxN = clamp01(rawFlux / fluxMax);
fluxBaseline = lerp(fluxBaseline, fluxN, 0.05);
const isSpike = fluxN > 0.18 &&
  fluxN > fluxBaseline * 1.8 &&
  spawnCooldown === 0;
```

Scanning each recognised utterance for command keywords; the latest word in a category wins:

```
const words = str.toLowerCase()
  .replace(/[^a-z0-9\s]/g, " ")
  .split(/\s+/);
for (const w of words) {
  if (Object.hasOwn(VOICE_COLORS, w))
    voiceState.bg = w;
  else if (VOICE_SHAPES.includes(w))
    voiceState.shape = w;
  // ...transport, volume, effects...
}
```